RSiena Post-Estimation Framework

Architecture & User Guide

Daniel Gotthardt

March 16, 2026

1 Overview

The RSiena post-estimation framework provides a unified pipeline for computing derived quantities from estimated Stochastic Actor-oriented Models (SAOMs). All quantities that go through the shared pipeline have three layers:

1. **Change contributions** (wide struct) — the matrix of objective function change statistics, one row per ego–choice (static) or ministep–choice (dynamic).
2. **Per-θ prediction functions** (`predictFun`) — compute the quantity of interest for a given parameter vector.
3. **Uncertainty orchestration** (`sienaPostestimate`) — draws $\theta^{(s)} \sim \mathcal{N}(\hat{\theta}, \hat{\Sigma}_\theta)$ and aggregates uncertainty.

Table 1 gives a complete inventory of post-estimation functions, their S3 method status, and whether they use the shared uncertainty pipeline.

Table 1: Post-estimation function inventory (current state).

Function	Class	`print`	`summary`	`plot`	Pipeline	Notes
`predict.sienaFit`	(data.frame)	–	–	–	Yes	
`marginalEffects`	(data.frame)	–	–	–	Yes	
`interpret_size / relativeImportance`	`sienaRI`	✓	✓	✓	Yes*	*optional
`entropy`	(data.frame)	–	–	–	Yes	
`selectionTable`	`selectionTable`	✓	–	–	No	
`influenceTable`	`influenceTable`	✓	–	–	No	

Two dispatch generics exist: `marginalEffects` (dispatches on `sienaFit` with full uncertainty, or `sienaEffects` for point-estimate only) and `entropy` (dispatches on `sienaFit` or `default`). The functions `interpret_selection` and `interpret_influence` are thin wrappers around `selectionTable` and `influenceTable`, respectively.

2 Quantities

2.1 Predicted Probabilities (`predict.sienaFit`)

For each ego–choice pair, the change probability is

$$P_j = \frac{\exp(u_j)}{\sum_h \exp(u_h)}, \qquad u_j = \sum_k \theta_k \, \delta_k(j),$$

where $\delta_k(j)$ are the change contributions for effect k and choice j. The `tieProb` variant adjusts for density direction: $r_{ij} = P_j$ if $x_{ij} = 0$ (creation), $r_{ij} = 1 - P_j$ if $x_{ij} = 1$ (dissolution).

2.2 Average Marginal Effects (`marginalEffects`)

The first difference uses one of two counterfactual update formulas, selected by perturbation mode:

Alter (one-alternative) mode. When the change statistic varies across alters (e.g. density, reciprocity):

$$P_j^{\text{cf}} = \frac{P_j \exp(\Delta u_j)}{1 - P_j + P_j \exp(\Delta u_j)}.$$

Ego-wide mode. When the change statistic is constant across alters (e.g. `egoX`):

$$P_j^{\text{cf}} = \frac{P_j \exp(\Delta u_j)}{\sum_k P_k \exp(\Delta u_k)}.$$

Here $\Delta u_j = \theta_e \delta_e$ for a unit change in effect e. The update formula is implemented in C++ (`mlogit_update`) and wrapped by the R helper `mlogit_update_r()`, which accepts string perturbation types (`"alter"`/`"ego"`).

Auto-detection is based on the `interactionType` column of the effects object (`"ego"` → ego-wide, otherwise alter). Users can override via `perturbType1`/`perturbType2`.

Second differences compose two such counterfactuals to measure interaction effects.

Mass contrasts. In ego-wide mode, the output includes two extra columns: `massCreation` (ΔP_i^+) and `massDissolution` (ΔP_i^-), summing dyad-level change-probability first differences within the creation and dissolution risk sets per ego.

Risk ratios. `mainEffect = "riskRatio"` returns P'/P instead of $P' - P$. Not fully tested yet for different perturbation types and second differences, but the same auto-detection should apply.

2.3 Relative Importance (`interpret_size`)

For each ego, the L^1 distance between the full-θ distribution and each leave-one-out distribution (effect k set to zero) measures the importance of effect k. Normalising gives relative importance:

$$RI_k(i) = \frac{\sum_j |P_j - P_j^{(-k)}|}{\sum_{k'} \sum_j |P_j - P_j^{(-k')}|}.$$

When `uncertainty = TRUE`, the calculation now runs through `sienaPostestimate`; the default path calls the legacy `sienaRI()` directly.

The return object has class `sienaRI` with `print`, `summary`, and `plot` methods (base R graphics: `barplot` for static, `matplot` for dynamic).

2.4 Entropy (`entropy`)

The normalised entropy (degree of certainty) is

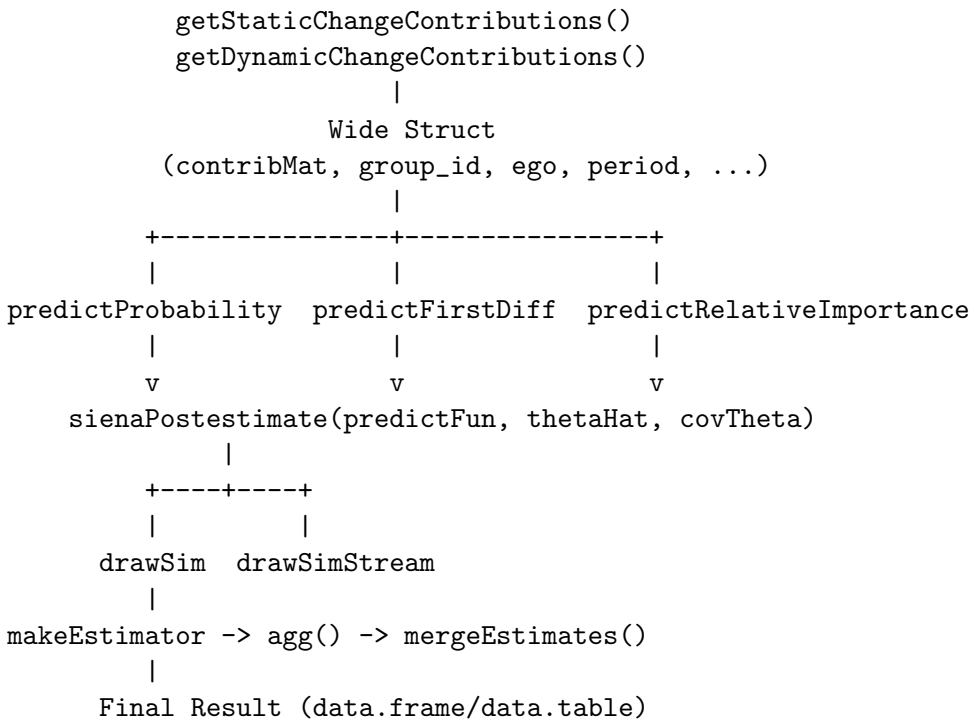
$$R_H = 1 - \frac{H}{\log J}, \qquad H = -\sum_j p_j \log p_j,$$

as defined in Snijders (2004) and now has a separate dispatch method. The `sienaFit` method computes R_H for each ego–choice pair, while the `default` method computes R_H for a single probability vector (e.g. the average distribution across egos). The former can be used to identify egos with more or less deterministic choice patterns; the latter can be used to summarise overall model fit (lower entropy indicates better fit).

2.5 Selection and Influence Tables

`selectionTable()` and `influenceTable()` compute interpretive summaries of network–behaviour co-evolution models. Both return typed data frames (`selectionTable`, `influenceTable`) with `print` methods but no `plot` or `summary`. They do not use the shared uncertainty pipeline.

3 Pipeline Architecture



4 Key Helper Functions

`calculateUtility(contribMat, theta)`
Matrix–vector product $\mathbf{s} \cdot \theta$.

`calculateChangeProb(utility, group_id)`
Group-wise softmax via `softmax_arma_by_group`.

`mlogit_update_r(p, delta_u, group_id, perturbType)`
R wrapper around the C++ `mlogit_update`; accepts `"alter"`/`"ego"` strings.

`computeMassContrasts(firstDiff, density, ego, period, group, type)`
Computes ΔP_i^+ and ΔP_i^- per ego × period.

`resolvePerturbType(effectName, effects)`
Auto-detects `"alter"` vs `"ego"` from `interactionType` metadata.

`makeEstimator(predictFun, agg, ...)`
Closure: `predictFun(theta)` → `agg()` → small result. Also aggregates mass-contrast columns when present.

`agg(outcomeName, data, level, condition, sum_fun)`
Flexible group-by aggregation (data.table fast path + base R fallback).

`alignThetaNoRate(theta, effectNames, ans)`
Aligns θ to effect names, dropping rate effects.

`groupColsList(pb)`
Extracts coordinate columns from the wide struct.

`attachContribColumns(out, effectNames, mat)`
Appends change-statistic columns to the output data.frame.

`mergeEstimates(pointEst, uncertainty)`
Joins point estimate with uncertainty summary.

5 Limitation: Network DVs Only

The shared pipeline currently supports only **network dependent variables**. Behaviour DVs (discrete and continuous) trigger a `stop()` in `marginalEffects`.

Continuous behaviour DVs in RSiena use an SDE model (Ornstein–Uhlenbeck with a Wiener process) rather than the multinomial-logit choice model. Marginal effects for continuous behaviour would require a different perturbation approach (shifting drift parameters rather than choice utilities).

6 Backward Compatibility

The original `sienaRI`, `sienaRIDynamics`, `print.sienaRI`, `summary.sienaRI`, `plot.sienaRI` remain fully functional. The default path in `interpret_size.sienaFit` still calls `sienaRI()` directly; the new path is activated by `uncertainty = TRUE`.

7 Outlook

The near-term priority is adding an S3 class (`"sienaMarginal"`) to the `marginalEffects` return value, together with `print`, `summary`, and `plot` methods (base R only, following `plot.sienaRI`). This would bring `marginalEffects` in line with the other postestimation tools that already carry typed classes.

Longer-term extensions include support for behaviour dependent variables (discrete and continuous) and period-level expected change counts that integrate the rate and choice components.

8 References

- Gotthardt, D. and Steglich, C.E.G. (2026). Marginal effects for the stochastic actor-oriented model.
- Indlekofer, N. and Brandes, U. (2013). Relative Importance of Effects in Stochastic Actor-oriented Models. *Network Science*, 1, 278–304.
- Snijders, T.A.B. (2004). Explained Variation in Dynamic Network Models. *Mathématiques et Sciences Humaines*, 168, 31–41.